

Progetto SN4M – Applicazioni Web e Cloud A.A. 2024/2025

Introduzione

Il progetto d'esame di laboratorio del corso di Applicazioni Web e Cloud 2024/2025 prevede la progettazione e realizzazione di un'applicazione web dedicata alla creazione e condivisione di playlist musicali: Social Network for Music (SN4M). Il progetto si propone di integrare competenze legate allo sviluppo web, all'accesso a sorgenti informative esterne (API), e alla gestione dell'interazione tra utenti in un contesto sociale online.

Gli obiettivi principali del progetto sono:

- Gestione delle playlist musicali personali, con la possibilità per ogni utente registrato di creare, modificare e cancellare playlist composte da brani musicali recuperati tramite le API REST di Spotify for developer.
- Gestione delle comunità musicali, dove ogni utente può cercare, creare o unirsi a comunità tematiche, contribuendo attivamente con contenuti musicali condivisi.
- Condivisione delle playlist all'interno delle comunità, rendendole visibili agli altri membri e abilitando meccanismi di interazione sociale come la visualizzazione dettagliata, il commento e l'importazione nel proprio profilo.
- Gestione dell'autenticazione utente, con registrazione, login e aggiornamento dei dati personali e delle preferenze musicali.
- Funzionalità di ricerca avanzata, che permettono agli utenti di esplorare le playlist condivise utilizzando criteri come tag descrittivi e brani contenuti.

L'applicazione web, che ho deciso di chiamare MyMusic, è un prototipo ed sviluppata in HTML5, CSS3 e JavaScript, mantenendo una netta separazione tra struttura e presentazione, e utilizzando il Web Storage del browser per la memorizzazione locale dei dati in formato JSON o XML. Inoltre, è progettata in modo adattivo per garantire un'esperienza utente ottimale anche su dispositivi con dimensioni di schermo differenti.

Salvataggio dati

Il salvataggio dei dati utente avviene utilizzando il Web Storage API, in particolare il `localStorage` del browser. Questo meccanismo consente di memorizzare dati strutturati lato client in formato JSON, rendendoli persistenti tra diverse sessioni di navigazione.

Durante la fase di registrazione, i dati inseriti dall'utente nel form vengono acquisiti e trasformati in un oggetto JavaScript contenente le seguenti informazioni:

- **ID utente** (`userID`)
- **Nome e cognome**
- **Genere**
- **Email**
- **Cantante preferito**
- **Genere musicale preferito**

- **Password:** salvata in formato hash MD5
- **Playlist associate:** inizialmente vuote

Questi dati vengono quindi recuperati dalla stringa di query URL tramite la funzione `getParameters()`, elaborati e aggregati in un oggetto `newUser`. L'oggetto viene poi inserito in un array di utenti (`usersArray`) precedentemente caricato da `localStorage`, oppure inizializzato se assente.

```
const usersArray = JSON.parse(localStorage.getItem('users')) || [];  
usersArray.push(newUser);  
localStorage.setItem('users', JSON.stringify(usersArray));
```

Questa operazione consente di serializzare l'intera lista degli utenti in formato JSON e memorizzarla nel browser sotto la chiave `"users"`. In questo modo, i dati sono accessibili in lettura e scrittura da qualsiasi altra pagina dell'applicazione che utilizzi lo stesso dominio, permettendo di implementare funzionalità come il login, il recupero delle playlist o la personalizzazione dell'interfaccia utente.

Progettazione dei macro-scenari

Registrazione e autenticazione

Tramite la home (`index.html`) dell'applicazione web, è possibile registrarsi sulla piattaforma, salvare le informazioni localmente tramite Web Storage e autenticarsi per accedere all'area riservata.

L'utente accede alla pagina di registrazione (`signup.html`) e compila un form con i propri dati anagrafici e musicali. Sono stati imposti dei controlli sui dati inseriti:

- **Nome e cognome:** almeno 3 caratteri massimo 25, solo lettere:

```
if (elem.id == "InputNome" || elem.id == "InputCognome") {  
    var espressioneRegolare = /^[a-zA-Z]{3,25}$/;  
    return espressioneRegolare.test(val) ? success(elem) : danger(elem);  
}
```

- **Genere:** campo selezione (M, F, Altro):

```
if (elem.id == "InputGenere") {  
    return (val != -1) ? success(elem) : danger(elem);  
}
```

- **UserID:** solo lettere minuscole, 4-10 caratteri

```
if (elem.id == "InputUID") {  
    var espressioneRegolare = /^[a-z]{4,10}$/;
```

```
    return espressioneRegolare.test(val) ? success(elem) : danger(elem);
}
```

- **Email e conferma email:** ci deve essere almeno un carattere prima e dopo la @, e tra 2 e 4 caratteri dopo il punto (esempio@gmail.com). La conferma email dev'essere uguale all'email.

```
if (elem.id == "InputEmail1") {
    var emailRegolare = /^[\w-\.\.]+@([\w-]+\.\.)+[\w-]{2,4}$/;
    return emailRegolare.test(val) ? success(elem) : danger(elem);
}
if (elem.id == "InputEmail2") {
    var email1 = document.getElementById("InputEmail1").value;
    return (email1 == val) ? success(elem) : danger(elem);
}
```

- **Password e conferma password:** ci devono essere almeno 8 caratteri, con almeno una cifra, una lettera minuscola, una maiuscola, e un carattere speciale. La conferma password dev'essere uguale alla password.

```
if (elem.id == "InputPassword1") {
    var passwordRegolare = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}$/;
    return passwordRegolare.test(val) ? success(elem) : danger(elem);
}
if (elem.id == "InputPassword2") {
    var password1 = document.getElementById("InputPassword1").value;
    return (password1 == val) ? success(elem) : danger(elem);
}
```

- **Artista e genere musicale preferiti:** (facoltativi).

Il sistema legge gli utenti già presenti in `localStorage` e verifica che lo `userID` inserito non sia già usato. In caso contrario, segnala l'errore sul campo. Inoltre, il checkbox deve essere selezionato. In caso contrario, compare un messaggio di errore sotto il campo.

Successivamente, viene fatta la conferma dei dati inseriti, i quali vengono passati e letti con la funzione `getParameters()`: che recupera ogni valore con `URLSearchParams`.

```
function getParameters() {
    var query = window.location.search;
    var parameters = new URLSearchParams(query);

    return {
        nome: parameters.get('nome'),
        cognome: parameters.get('cognome'),
        genere: parameters.get('genere'),
        userID: parameters.get('uid'),
    };
}
```

```
    email: parameters.get('email'),
    cantantePreferito: parameters.get('preferenza'),
    generePreferito: parameters.get('generePreferito'),
    password: parameters.get('password')
  };
}
```

Quando l'utente conferma `createUser()` crea e memorizza l'oggetto utente che verrà aggiunto all'array `users`:

```
function createUser() {
  var params = getParameters();
  const newUser = {
    userID: params.userID,
    nome: params.nome,
    cognome: params.cognome,
    genere: params.genere,
    email: params.email,
    cantantePreferito: params.cantantePreferito,
    generePreferito: params.generePreferito,
    password: CryptoJS.MD5(params.password).toString(),
    playlist: [],
  };
  const usersArray = JSON.parse(localStorage.getItem('users')) || [];
  usersArray.push(newUser);
  window.localStorage.setItem('users', JSON.stringify(usersArray));
  window.location.href = "regOk.html";
}
```

Infine, l'utente può accedere al proprio profilo utente (`login.html`) inserendo userID e password, e verificando che le credenziali siano salvate nel `localStorage`. I vari controlli delle credenziali vengono gestiti tramite `controlla()`. In particolare, se lo userID esiste, viene letto il valore del campo password e convertita in hash MD5:

```
password = CryptoJS.MD5(password).toString();
```

In questo modo, il valore hash viene confrontato con quello memorizzato durante la registrazione.

Una volta effettuato il login, il sistema salva nel `sessionStorage` l'informazione e l'utente viene indirizzato alla home (`index.html`):

```
if (utenti[tmp].password == password) {
  sessionStorage.setItem('logged', username);
  window.location.href = 'index.html';
}
```

Gestione profilo utente

Nel momento in cui l'utente visita la pagina di profilo si cerca lo userID tra quelli presenti nel `localStorage`, e se esiste vengono mostrate le informazioni personali. Inoltre, tramite il `sessionStorage`, il sistema confronta l'utente del profilo con quello attualmente loggato. Se l'utente è il proprietario del profilo, vengono mostrati (oltre le informazioni personali) i pulsanti modifica ed elimina; se non lo è vengono mostrate solo le informazioni relative a quel profilo.

Se l'utente vuole modificare le proprie informazioni, cliccando il relativo bottone, viene indirizzato a `edit-profile.html`. Se l'utente vuole eliminare il proprio profilo, cliccando il relativo bottone, l'utente viene rimosso dal `localStorage`, vengono eliminate tutte le playlist di cui l'utente era autore, la sessione viene rimossa dal `sessionStorage`, e viene reindirizzato alla home (`index.html`).

Gestione delle playlist

La funzionalità di gestione delle playlist è accessibile esclusivamente agli utenti autenticati e viene implementata attraverso la pagina `playlist.html`. Essa consente la visualizzazione dei dettagli della playlist, la gestione dinamica dei brani (aggiunta e rimozione), nonché l'eliminazione completa della playlist.

L'autenticazione dell'utente viene verificata tramite `sessionStorage.getItem('logged')`, mentre i dati dell'utente corrispondente vengono recuperati dal `localStorage`. Se l'utente non risulta autenticato, l'accesso alla pagina viene negato. Per ogni playlist, viene mostrato un elenco dettagliato dei brani in essa contenuti. Per ciascun brano sono visualizzati: immagine di copertina, titolo, artista e album. Una volta creata, la playlist può essere modificata dinamicamente attraverso l'aggiunta o la rimozione dei brani.

Nel caso in cui l'utente tenti di rimuovere l'ultimo brano presente nella playlist, il sistema mostra un avviso informando che la playlist verrà eliminata completamente. Questa scelta implementativa è giustificata dall'idea che una playlist vuota non abbia significato funzionale e occupi memoria inutilmente. Quando una playlist viene eliminata (manualmente o perché svuotata), essa viene rimossa sia dall'array `user.playlist` che dal `localStorage`, assicurando la consistenza e l'aggiornamento dello stato dell'applicazione.

Gestione delle comunità musicali

La gestione delle comunità musicali viene mostrata tramite `communities.html`. L'accesso a questa funzionalità è riservato agli utenti autenticati, il cui stato viene verificato tramite `sessionStorage.getItem('logged')`; in caso contrario, l'interfaccia mostra un messaggio che invita l'utente ad effettuare il login.

Le comunità vengono recuperate dal `localStorage` e mostrate. L'utente può cercare una comunità per titolo, descrizione, tag associati, e per ogni comunità vengono visualizzati titolo, descrizione, tag, numero di membri, e stato di iscrizione (se l'utente è già iscritto compare `sei iscritto`). Ogni scheda di comunità mostra un pulsante: `unisciti` (se non è ancora membro) oppure `disiscriviti` (se l'utente è già iscritto). In caso di disiscrizione, viene mostrato un messaggio di conferma tramite `SweetAlert`, una libreria JavaScript che consente di mostrare finestre di dialogo user-friendly, in sostituzione ai classici `alert()`, `confirm()` e `prompt()` del browser; se l'utente conferma, viene rimosso dall'elenco dei membri.

Alla creazione di una comunità viene creato un nuovo oggetto comunità con ID univoco (`Date.now() + Math.random()`), l'autore della comunità viene inserito automaticamente come primo membro, e la comunità viene salvata nel `localStorage`.

Gestione delle condivisioni

La condivisione delle playlist avviene tramite `sharing.html` ed è accessibile solo agli utenti autenticati, verificati tramite `sessionStorage.getItem('logged')`. Il codice `sharing.js` gestisce la selezione delle playlist da condividere, la scelta della comunità di destinazione, l'inserimento di tag e descrizione della condivisione, e il salvataggio del `localStorage` sotto la struttura delle comunità.

Alla visualizzazione della pagina dell'utente loggato vengono caricate e mostrate in un menù a tendina (`playlist-select`), e le comunità a cui un utente è iscritto vengono recuperate dal `localStorage` e mostrate in un secondo menù (`community-select`). Quando l'utente preme condividi, il sistema legge la playlist selezionata, la comunità scelta, i tag e la descrizione. Viene verificato che entrambi i campi siano selezionati, che la playlist esista, e che la comunità sia valida.

Inoltre, prima di salvare la condivisione, viene fatto il controllo che l'utente non abbia già condiviso quella stessa playlist nella stessa comunità. Se tutti i controlli sono superati, la playlist viene salvata all'interno del campo `sharedPlaylists` della comunità, i dati vengono salvati nel `localStorage` e viene mostrato un messaggio di conferma.

Considerazioni finali

Il progetto, pur essendo un prototipo, implementa numerose funzionalità proprie di un social network musicale, con attenzione alla user experience, alla validazione dei dati e alla persistenza delle informazioni lato client. Il sistema è progettato per essere estensibile, e potrebbe in futuro integrare autenticazione tramite OAuth con Spotify, commenti tra utenti o funzionalità server-side.